

AAP — Spécification technique Phase 1

1. Backend — fondations avant le CRUD

Sécurité

Avant toute fonctionnalité métier : JWT access/refresh tokens ; OAuth 2.0 / Google ; vérification email ; hashage des mots de passe avec Argon2 ou bcrypt ; CORS strict ; CSRF selon le mode d'authentification ; rate limiting ; protection brute-force ; validation des entrées ; RBAC ; séparation utilisateur/admin ; sécurité des headers ; gestion centralisée des exceptions ; logs de sécurité ; audit logs ; secrets via variables d'environnement ; aucune donnée sensible dans les logs ; pagination des endpoints ; limites de payloads ; contrôle des uploads ; HTTPS en production ; cookies HttpOnly, Secure et SameSite ; rotation/révocation des refresh tokens.

Redis

Redis sert au rate limiting, cache, sessions/token blacklist si nécessaire, données temporaires, OTP/email verification codes et tâches courtes. Celery pourra l'utiliser plus tard.

Frontend

```
↓
Traefik
Django / DRF
↓
Redis
↓
PostgreSQL
```

Redis ne remplace pas PostgreSQL. PostgreSQL reste la source de vérité.

2. Principe fondamental : le frontend ne possède pas les données métier

Le frontend ne doit pas contenir les données administrables : nom de l'application, description, images, plans,

rôles, permissions, contenu des pages, messages métier ou configuration. Le frontend devient principalement un client de l'API. Les éléments purement techniques peuvent rester dans le code, mais les textes devraient idéalement passer par i18n.

3. Hashing et chiffrement

Les mots de passe sont hashés avec Argon2 ou bcrypt et ne doivent pas être récupérables. Les données récupérables mais sensibles, comme customer database credentials, API keys et OAuth secrets, devront être chiffrées et non hashées.

4. Base de données

PostgreSQL est la base principale avec Django ORM. Les données persistantes sont dans PostgreSQL ; Redis est une couche complémentaire de cache et de données temporaires.

5. Architecture Django

backend/

```
... config/
... apps/
.... accounts/
```

- ■■■ Icon/
- ■■■ Breadcrumb/
- ■■■ Sidebar/
- ■■■ Navbar/
- ■■■ Modal/
- ■■■ Loader/
- ■■■ EmptyState/
- ■■■ ErrorState/
- ■■■ Pagination/
- ■■■ DataTable/
- ■■■ Charts/
- ■■■ LanguageSwitcher/
- ■■■ ThemeSwitcher/
- pages/
- layouts/
- features/
- services/
- store/
- hooks/
- routes/
- i18n/
- types/
- utils/
- assets/

12. Redux Toolkit

Redux Toolkit gère surtout l'état UI/global : thème, langue, authentification/permissions et sidebar. Les données serveur pourront être gérées avec TanStack Query plutôt que de mettre toutes les données API dans Redux.

13. i18n

```
i18n/
■■■ fr/
■ ■■■ translation.json
■■■ en/
    ■■■ translation.json
```

14. Charts

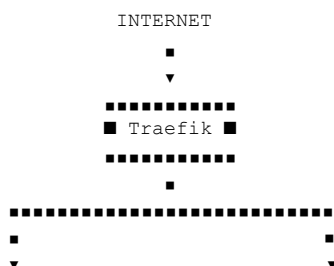
Pour AAP, utiliser Apache ECharts plutôt que Recharts. Les graphiques seront encapsulés dans reusedComponents/Charts.

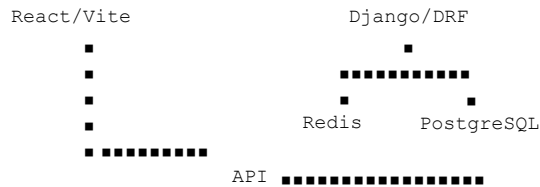
```
reusedComponents/Charts/
■■■ LineChart/
■■■ BarChart/
■■■ AreaChart/
■■■ PieChart/
■■■ ChartCard/
```

15. Pages initiales

Public : Home, Login, Register, Verify Email, Forgot Password. User : Dashboard, Profile, Settings. Admin : Dashboard, Users, User details, Notifications, Audit.

16. Architecture complète de la première version

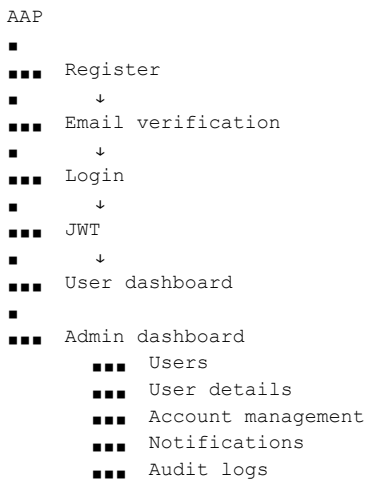




17. Ce qu'on ne fait pas maintenant

Phase1 : React,Vite, TypeScript, Tailwind, Redux Toolkit,i18n, ECharts, Django, DRF, JWT, Google OAuth, PostgreSQL et Redis. Phase 2 : Docker, Traefik, Celery, CI/CD et GitHub Actions. Phase 3 : LLM, Ollama, RAG, embeddings, vector DB, tool calling et MCP. Phase 4 : connexions aux bases clientes, monitoring, logs, génération SQL en lecture seule, diagnostics IA et observabilité.

18. Première cible



Objectif : une base solide avec Security + PostgreSQL + Redis + Django REST + JWT/OAuth + React/TypeScript + Redux Toolkit + i18n + composants réutilisables, sans paiement, marketplace ou IA dans cette première étape.